

# OBJECT ORIENTED PROGRAMMING USING C++



**Study Materials**

## Linear Search:

- Linear search (also known as sequential search) is the simplest searching algorithm. It sequentially checks each element in a list until a match is found or the entire list has been traversed.
- Data Requirement: Can be used on both sorted and unsorted data.
- Time Complexity:
  - Best Case:  $O(1)$  (target is the first element).
  - Average & Worst Case:  $O(n)$  (target is in the middle, at the end, or not present), where  $n$  is the number of elements.
  - Space Complexity:  $O(1)$  (constant extra space).

## Linear Search:

- Start from index 0
- Compare current element with target
- If match → return index
- Else → move to next index
- If end reached → element not found

## Linear Search – Complexity:

- Best Case:  $O(1)$  (element at first position)
- Average Case:  $O(n)$
- Worst Case:  $O(n)$  (element at last / not present)
- Space Complexity:  $O(1)$

## Implementation Binary Search Algorithm:

- The Binary Search Algorithm can be implemented in the following two ways
- Iterative Binary Search Algorithm
- Recursive Binary Search Algorithm
- Analysis:
  - Time Complexity:  $O(\log N)$
  - Auxiliary Space:  $O(1)$

## Example:

- For a binary search to work, it is mandatory for the target array to be sorted. We shall learn the process of binary search with a pictorial example.
- The following is our sorted array and let us assume that we need to search the location of value 31 using binary search.



| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
|----|----|----|----|----|----|----|----|----|----|
| 10 | 14 | 19 | 26 | 27 | 31 | 33 | 35 | 42 | 44 |

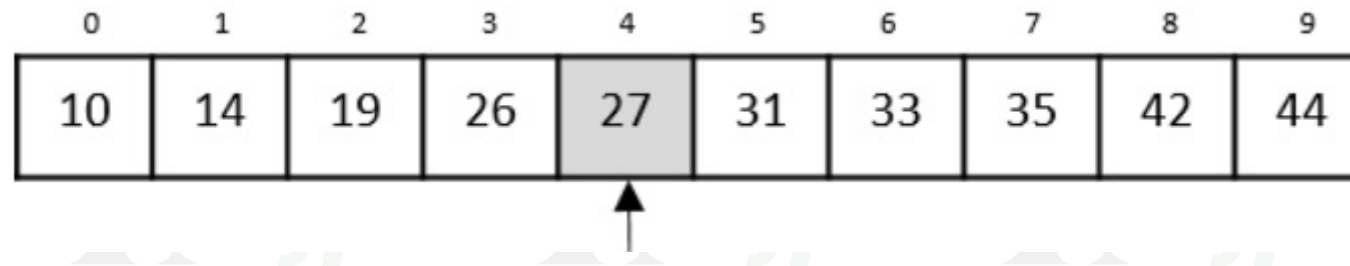
- First, we shall determine half of the array by using this formula –

```
mid = low + (high - low) / 2
```

## Example:

- Here it is,  $0 + (9 - 0) / 2 = 4$  (integer value of 4.5). So, 4 is the mid of the array.

|    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|
| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
| 10 | 14 | 19 | 26 | 27 | 31 | 33 | 35 | 42 | 44 |



- Now we compare the value stored at location 4, with the value being searched, 31. We find that the value at location 4 is 27, which is not a match.
- As the value is greater than 27 and we have a sorted array, so we also know that the target value must be in the upper portion of the array.–

## Example:

- We change our low to mid + 1 and find the new mid value again.

```
low = mid + 1  
mid = low + (high - low) / 2
```

- Our new mid is 7 now. We compare the value stored at location 7 with our target value 31.

| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
|----|----|----|----|----|----|----|----|----|----|
| 10 | 14 | 19 | 26 | 27 | 31 | 33 | 35 | 42 | 44 |





## Example:

- The value stored at location 7 is not a match, rather it is less than what we are looking for. So, the value must be in the lower part from this location.

| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
|----|----|----|----|----|----|----|----|----|----|
| 10 | 14 | 19 | 26 | 27 | 31 | 33 | 35 | 42 | 44 |

- Hence, we calculate the mid again. This time it is 5.

| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
|----|----|----|----|----|----|----|----|----|----|
| 10 | 14 | 19 | 26 | 27 | 31 | 33 | 35 | 42 | 44 |



## Example:

- We compare the value stored at location 5 with our target value. We find that it is a match.

| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
|----|----|----|----|----|----|----|----|----|----|
| 10 | 14 | 19 | 26 | 27 | 31 | 33 | 35 | 42 | 44 |

- We conclude that the target value 31 is stored at location 5.

## Linear vs Binary Search Comparison:

| Feature         | Linear Search | Binary Search    |
|-----------------|---------------|------------------|
| Data Order      | Any           | Sorted only      |
| Time Complexity | $O(n)$        | $O(\log n)$      |
| Speed           | Slow          | Fast             |
| Implementation  | Simple        | Slightly complex |

## Asymptotic Notations:

- Asymptotic Notations are mathematical tools used to analyze the performance of algorithms by understanding how their efficiency changes as the input size grows.
- These notations provide a concise way to express the behavior of an algorithm's time or space complexity as the input size approaches infinity.
- Rather than comparing algorithms directly, asymptotic analysis focuses on understanding the relative growth rates of algorithms' complexities.
- It enables comparisons of algorithms' efficiency by abstracting away machine-specific constants and implementation details, focusing instead on fundamental trends.
- Asymptotic analysis allows for the comparison of algorithms' space and time complexities by examining their performance characteristics as the input size varies.

## Asymptotic Notations:

- Asymptotic notations describe the efficiency of an algorithm as the input size grows.
- Three common notations:
  - Big-O ( $O$ ): Upper bound (worst-case scenario).
  - Omega ( $\Omega$ ): Lower bound (best-case scenario).
  - Theta ( $\Theta$ ): Tight bound (average-case scenario).

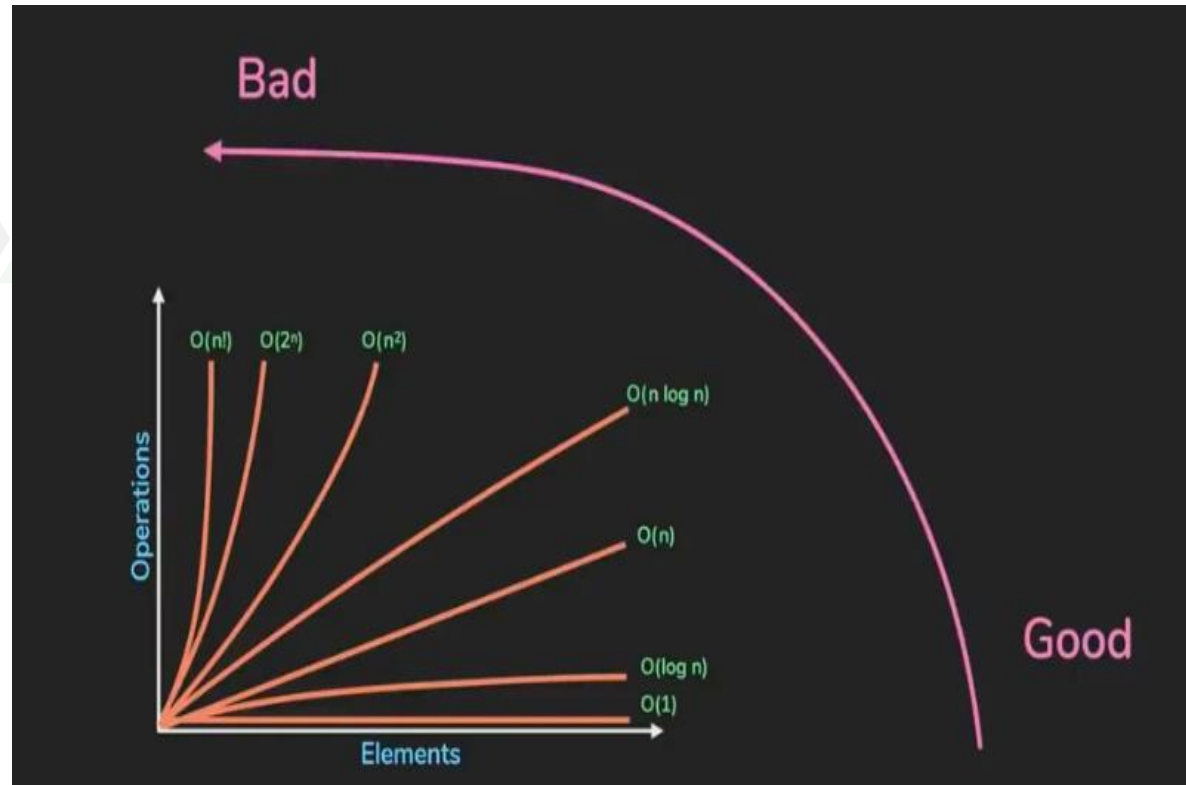
## Big O notation (O):

- It is defined as upper bound and upper bound on an algorithm is the most amount of time required ( the worst case performance).
- Big O notation is used to describe the asymptotic upper bound.
- Mathematically, if  $f(n)$  describes the running time of an algorithm;  $f(n)$  is  $O(g(n))$  if there exist positive constant  $C$  and  $n_0$  such that,

$$0 \leq f(n) \leq Cg(n) \text{ for all } n \geq n_0$$

- $n$  = used to give upper bound a function.
- If a function is  $O(n)$ , it is automatically  $O(n\text{-square})$  as well.

## Representation of Big O Notation :



## Big Omega notation ( $\Omega$ ) :

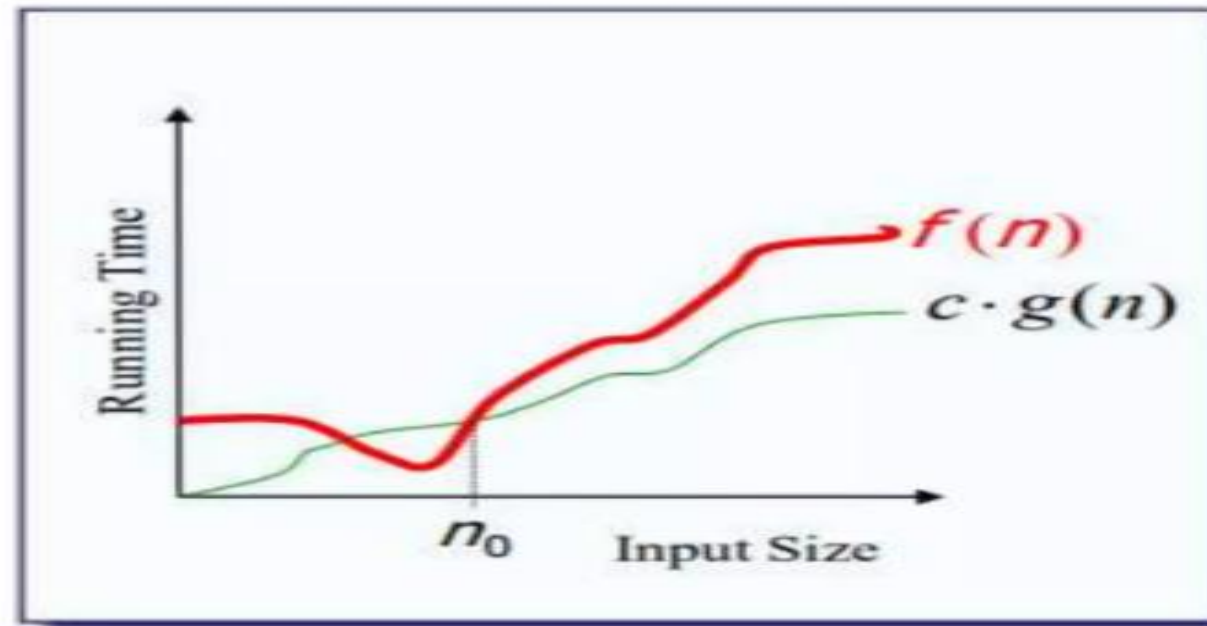
- It is define as lower bound and lower bound on an algorithm is the least amount of time required ( the most efficient way possible, in other words best case).
- Just like O notation provide an asymptotic upper bound,  $\Omega$ notation provides asymptotic lower bound.
- Let  $f(n)$  define running time of an algorithm;
- $f(n)$  is said to be  $\Omega(g(n))$  if there exists positive constant C and  $(n_0)$  such that

$$0 \leq Cg(n) \leq f(n) \text{ for all } n \geq n_0$$

- $n$  = used to given lower bound on a function
- If a function is  $\Omega(n\text{-square})$  it is automatically  $\Omega(n)$  as well.



Big Omega notation ( $\Omega$ ) :



## Big Theta notation ( $\Theta$ ) :

- It is define as tightest bound and tightest bound is the best of all the worst case times that the algorithm can take.
- Let  $f(n)$  define running time of an algorithm.
- $f(n)$  is said to be  $\Theta(g(n))$  if  $f(n)$  is  $O(g(n))$  and  $f(n)$  is  $\Omega(g(n))$ .
- The equation simply means there exist positive constants  $C1$  and  $C2$  such that  $f(n)$  is sandwich between  $C2 g(n)$  and  $C1g(n)$ .

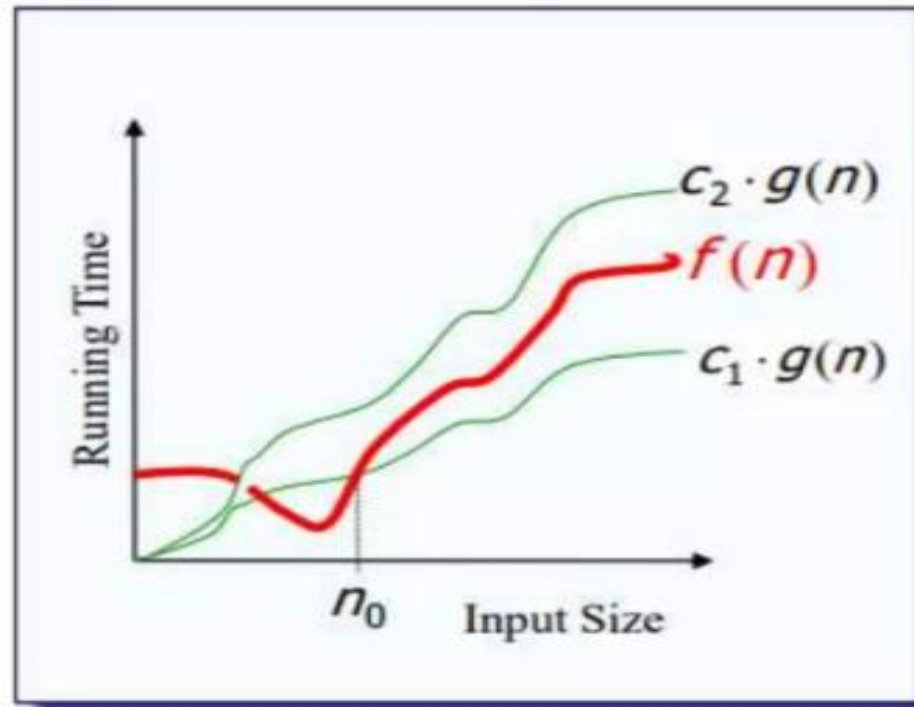
$$0 \leq f(n) \leq C1g(n) \text{ for } n \geq n_0$$

$$0 \leq C2g(n) \leq f(n) \text{ for } n \geq n_0$$

*Merging both the equation, we get :*

$$0 \leq C2g(n) \leq f(n) \leq C1g(n) \text{ for } n \geq n_0$$

## Big Theta notation ( $\Theta$ ) :



## Difference Between Big oh, Big Omega and Big Theta :

| S.No. | Big O                                                                                                                                                          | Big Omega ( $\Omega$ )                                                                                                                                         | Big Theta ( $\Theta$ )                                                                                                                        |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 1.    | It is like ( $\leq$ )<br>rate of growth of an algorithm is less than or equal to a specific value.                                                             | It is like ( $\geq$ )<br>rate of growth is greater than or equal to a specified value.                                                                         | It is like ( $=$ )<br>meaning the rate of growth is equal to a specified value.                                                               |
| 2.    | The upper bound of algorithm is represented by Big O notation. Only the above function is bounded by Big O. Asymptotic upper bound is given by Big O notation. | The algorithm's lower bound is represented by Omega notation. The asymptotic lower bound is given by Omega notation.                                           | The bounding of function from above and below is represented by theta notation. The exact asymptotic behavior is done by this theta notation. |
| 3.    | Big O – Upper Bound                                                                                                                                            | Big Omega ( $\Omega$ ) – Lower Bound                                                                                                                           | Big Theta ( $\Theta$ ) – Tight Bound                                                                                                          |
| 4.    | It is define as upper bound and upper bound on an algorithm is the most amount of time required ( the worst case performance).                                 | It is define as lower bound and lower bound on an algorithm is the least amount of time required ( the most efficient way possible, in other words best case). | It is define as tightest bound and tightest bound is the best of all the worst case times that the algorithm can take.                        |
| 5.    | Mathematically: Big Oh is $0 \leq f(n) \leq Cg(n)$ for all $n \geq n_0$                                                                                        | Mathematically: Big Omega is $0 \leq Cg(n) \leq f(n)$ for all $n \geq n_0$                                                                                     | Mathematically – Big Theta is $0 \leq C_2g(n) \leq f(n) \leq C_1g(n)$ for $n \geq n_0$                                                        |